MaskGuard AI

Face Mask Detection System

Feasibility Study Report

Computer Vision Lab | Final Project Submission

Prepared by:

Data Fardeen | Amina Asghar | Iman Humayun

GitHub Repository:

https://github.com/Fardeen37/Face-Mask-Detection-System

1. Introduction

MaskGuard AI is an intelligent face mask detection system built on computer vision and deep learning techniques. The final version of this system, referred to throughout this document, represents a substantial upgrade from the mid-project submission that was demonstrated earlier in the semester. The system was developed incrementally across the Computer Vision Lab sessions, with each lab contributing new techniques that were integrated directly into the working codebase.

The system is capable of automatically identifying whether individuals in static images, live webcam feeds, or uploaded video files are wearing face masks correctly, incorrectly, or not at all. Every detected face receives a colour-coded bounding box, a confidence score, and a mask coverage percentage, all generated and displayed in real time.

The mid-project version of this system relied on a straightforward Histogram of Oriented Gradients (HOG) feature extraction pipeline paired with a Support Vector Machine (SVM) classifier. While this formed a functional baseline, it had clear limitations in terms of accuracy, robustness, and usability. The final system addresses all of those limitations. Feature representation has been expanded to a fused vector combining HOG, Gabor texture filters, and HSV colour histograms. This richer representation feeds into a research-grade pipeline of StandardScaler, PCA, and SVM. Alongside this classical pipeline, a fine-tuned MobileNetV2 deep learning model serves as the primary high-accuracy classifier, selectable at runtime through a radio button in the interface.

Additional capabilities introduced in the final version include a Content-Based Image Retrieval (CBIR) explainability tab, an Auto-Night Mode that applies adaptive gamma correction for low-light environments, morphological mask-coverage analysis on every detected face, and a video file processing pipeline. All functionality is packaged into a four-tab Gradio web interface that can be shared publicly via a generated link without any installation required from the end user.

2. Problem Statement

The core problem this system addresses has remained consistent from the beginning of the project: manual monitoring of face mask compliance in high-traffic environments is labour-intensive, inconsistent, and not scalable. A single human monitor cannot reliably track mask usage across a busy hospital entrance, transport hub, or factory floor at all times. The need for an automated, real-time solution is clear.

The final project goes further by addressing two specific technical challenges that emerged during development and were not handled in the mid-project version.

The first challenge is class imbalance in the training data. The Kaggle face-mask-detection dataset contains 3,232 annotated instances of faces with masks, 717 instances without masks, and only 123 instances of masks worn incorrectly. A classifier trained naively on this data would simply learn to always predict the majority class and still achieve over 80% accuracy on paper, while completely failing on the minority classes that matter most for compliance monitoring. The mid-project SVM baseline exhibited this exact behaviour. The final system addresses this through weighted loss functions in MobileNetV2 training and the class_weight balanced setting in the SVM, ensuring the minority classes are given appropriate emphasis during training.

The second challenge is the false-positive problem in face detection. The mid-project pipeline attempted to run four Haar and LBP cascade classifiers simultaneously in an effort to maximise recall. In practice, this caused the system to detect faces in background objects such as picture frames, ventilation grilles, and other rectangular patterns, flooding the classification stage with incorrect inputs and also slowing webcam processing to an unusable frame rate. The final system rationalises face detection to two cascades, haarcascade_frontalface_alt2 as the primary detector and haarcascade_frontalface_default as the fallback, both operating on CLAHE-preprocessed frames. A user-adjustable sensitivity slider controls the trade-off between recall and precision. This resolves both the false-positive problem and the frame rate degradation without sacrificing meaningful detection coverage.

3. Objectives of the System

The following objectives define the complete scope of the final system. Each bullet point represents a distinct capability that the deployed application must deliver:

- Detect human faces in static images, real-time webcam streams, and uploaded video files using CLAHE-enhanced Haar Cascade classifiers with an adjustable sensitivity parameter that lets the user tune the trade-off between detection recall and false positives.
- Classify each detected face into one of three categories (With Mask, Without Mask, or Mask Worn Incorrectly) using either a fine-tuned MobileNetV2 convolutional neural network or a HOG plus Gabor plus PCA plus SVM pipeline, with the model choice selectable at runtime through the interface without restarting the system.
- Estimate mask coverage percentage on each detected face using morphological image analysis that includes HSV colour segmentation, morphological close and open operations, dilation, and contour detection, with the calculated percentage displayed alongside each bounding box.
- Explain predictions transparently through a dedicated CBIR tab that retrieves the top three most visually similar training examples from the 4,072-patch index for any uploaded face image, displaying them side by side with their class labels and similarity percentages.
- Handle dark or low-light environments through an Auto-Night Mode that measures mean frame brightness and applies adaptive gamma correction via a lookup table before face detection runs, activating only when the frame falls below a defined brightness threshold to avoid unnecessary processing overhead.
- Process uploaded video files frame by frame at a configurable skip rate, annotate each frame with detection results, export the annotated video as a downloadable file, and generate an overall mask compliance summary report covering all faces detected across the video.
- Present all system functionality through a clean, four-tab Gradio web interface that requires no coding knowledge from the end user and generates a public shareable link on launch so the system can be accessed from any browser on any device.

4. Technical Feasibility

The final system is built entirely on mature, well-documented, open-source libraries. The architecture intentionally combines classical computer vision with deep learning, giving users the ability to switch between both approaches at runtime. This section maps every system requirement to the exact tools, libraries, and configuration values used in the final version.

System Requirement	Tools / Libraries / Configuration
Image preprocessing and enhancement	OpenCV (cv2), NumPy, CLAHE via cv2.createCLAHE with clipLimit 2.0 and tileGridSize 8x8
Face detection	Haar Cascade haarcascade_frontalface_alt2 as primary detector plus haarcascade_frontalface_default as fallback; tunable sensitivity slider exposed in Gradio interface
HOG feature extraction	scikit-image hog() with 9 orientations, 8x8 pixels per cell, 2x2 cells per block, feature_vector=True; produces approximately 1764-dimensional descriptor per face patch
Gabor texture extraction (Lab 07)	OpenCV cv2.getGaborKernel with 4 orientations and 4 frequencies producing 16 kernels total; mean and standard deviation per kernel concatenated into a 32-dimensional descriptor
HSV colour histogram	OpenCV cv2.calcHist on HSV channels with 8x8x8 bins, normalised to a 512-dimensional vector per face patch
Feature fusion vector	HOG descriptor (~1764 dims) + Gabor descriptor (32 dims) + colour histogram (512 dims) concatenated into a single approximately 1796-dimensional fused vector per patch

System Requirement	Tools / Libraries / Configuration
PCA dimensionality reduction (Labs 11 and 12)	sklearn PCA with 100 principal components; compresses the fused vector from approximately 1796 to 100 dimensions, retaining sufficient variance while improving SVM training speed and generalisation
SVM classification pipeline	sklearn Pipeline wrapping StandardScaler then PCA(100) then SVC with rbf kernel, C=10, class_weight=balanced, probability=True for confidence scores
CBIR visual retrieval (Lab 08)	scipy.spatial.distance.cdist using cosine distance; descriptor index built from colour histogram (512 dims) plus Gabor features (32 dims) across all 4072 training patches
Deep learning model	PyTorch MobileNetV2 pretrained on ImageNet; first 14 feature extraction layers frozen; custom three-class classification head appended
CNN training and fine-tuning	AdamW optimiser with lr=5e-4 and weight_decay=1e-3; CosineAnnealingLR scheduler with T_max=20; Weighted CrossEntropyLoss with class weights [1.0, 2.5, 2.0]; 20 epochs at batch size 32
Data augmentation	torchvision transforms: RandomHorizontalFlip, RandomRotation(15 degrees), ColorJitter adjusting brightness, contrast, and saturation; all images resized to 128x128 before processing
Morphological mask-coverage analysis	OpenCV HSV segmentation targeting light, low-saturation regions; MORPH_CLOSE and MORPH_OPEN for noise removal; dilation to expand detected region; contour detection to compute coverage percentage per face

System Requirement	Tools / Libraries / Configuration
Auto-Night Mode	OpenCV cv2.LUT with adaptive gamma ranging from 1.1 at brightness threshold 80 up to 2.5 for very dark frames; step bypassed entirely when mean frame brightness exceeds threshold
Dataset handling	Kaggle API downloading andrewmvd/face-mask-detection (~150 MB); custom PASCAL VOC XML parser; 853 images producing 4072 face patches across three label classes
GPU acceleration	Google Colab NVIDIA T4 GPU via CUDA for MobileNetV2 training; SVM pipeline runs entirely on CPU
Web interface and deployment	Gradio 4.44.0 and above with four tabs (Webcam, Image, Video, CBIR); custom teal dark theme using DM Mono font; public shareable link generated on launch
Performance evaluation	scikit-learn classification_report and confusion_matrix; per-class precision, recall, and F1 score reported for all three label categories
Visualisation	Matplotlib for training curves; OpenCV annotation overlays for bounding boxes, confidence labels, and mask coverage text on all output frames

4.1 Dual-Model Architecture

One of the more significant design decisions in the final system is the ability to switch between two classifiers at runtime without restarting the application. The Gradio interface exposes a radio button that selects either the MobileNetV2 CNN or the HOG plus Gabor plus PCA plus SVM pipeline. Both models are loaded into memory at startup and remain ready simultaneously. The SVM pipeline is always used for CBIR explanations regardless of which model is selected for live detection.

The MobileNetV2 model is loaded with pretrained ImageNet weights. The first 14 feature extraction layers are frozen during training, meaning their weights do not change. Only the remaining layers and the custom three-class classification head are updated during fine-tuning. This partial freezing approach keeps training time manageable on a Colab T4 GPU while still allowing the model to adapt high-level feature detectors to the specific visual characteristics of masked and unmasked faces. The custom head consists of a Dropout layer with a rate of 0.4, a 256-unit hidden linear layer, a ReLU activation, a second Dropout layer with a rate of 0.3, and a final three-class linear output.

The SVM pipeline wraps all feature engineering steps, including standardisation, PCA, and classification, into a single sklearn Pipeline object. This design makes saving, loading, and running inference consistent and straightforward, since the entire pipeline is serialised and deserialised as a single object. The PCA step was added specifically because the raw fused feature vector was approximately 1,796 dimensions, and SVM training and inference performance typically degrades significantly on high-dimensional data without dimensionality reduction. Compressing to 100 principal components retained sufficient variance to preserve classification accuracy while making the SVM considerably faster to train and evaluate.

4.2 CBIR Visual Retrieval System (Lab 08)

The CBIR tab functions as the system's explainability layer. When a user uploads a face image to this tab, the system classifies it using the SVM pipeline and simultaneously retrieves the top three most visually similar training patches from the full 4,072-patch index. Similarity is computed using cosine distance on a descriptor that combines the HSV colour histogram (512 dimensions) and Gabor texture features (32 dimensions). The resulting visual panel displays the query image on the left, followed by the three nearest neighbours, each labelled with their class name and similarity percentage.

The 4,072-patch index was built by extracting and storing the colour plus Gabor descriptor for every face patch in the training dataset across all three label classes. At query time, the descriptor of the uploaded image is compared against all stored descriptors using cdist from scipy, and the three lowest cosine distances are returned as the most similar matches.

This feature matters for a real-world compliance system because it makes the model's reasoning visible to a non-technical user. A security supervisor who receives an unexpected classification result can open the CBIR tab, upload the face image in question, and immediately see which training examples the system found most similar. This builds confidence in correct predictions and provides a clear path for manual verification when results seem questionable. It achieves explainable AI entirely from classical computer vision techniques, with no separate explainability library or post-hoc analysis tool required.

4.3 Auto-Night Mode and Morphological Analysis

The Auto-Night Mode was added after testing revealed that dark webcam environments were causing face detection to fail before the classification stage even had a chance to run. When ambient lighting is poor, the contrast in the frame is insufficient for the Haar Cascade to locate faces reliably. The solution is a LUT-based adaptive gamma correction step applied to each webcam frame before face detection.

The mean brightness of the frame is measured as a single value. If this mean falls below a threshold of 80 out of 255, gamma is computed proportionally based on how far the brightness is below the threshold. Darker frames receive stronger correction. At the threshold boundary, gamma begins at 1.1 (a modest brightening). For very dark frames approaching a mean brightness of zero, gamma reaches 2.5 (a strong brightening). Frames with mean brightness above 80 bypass this step entirely, avoiding unnecessary processing overhead in well-lit environments. The corrected frame is then passed to the Haar Cascade detector.

Morphological mask-coverage analysis runs on every detected face regardless of which classification model is selected. The face patch is first converted from BGR to HSV colour space. A colour range corresponding to typical mask materials, which tend to appear as light, low-saturation regions in HSV, is thresholded into a binary mask image. Morphological close and open operations are applied to remove small noise regions and fill gaps in the detected area. A dilation step then expands the detected region slightly to account for mask edges. Contours are drawn on the output frame, and the proportion of the face region covered by the detected mask area is computed and displayed alongside each bounding box as a percentage value.

5. Operational Feasibility

The final system is operationally more capable than the mid-project version across every deployment scenario. The four-tab Gradio interface covers the main use cases a real-world operator would need, structured so that each tab addresses a distinct workflow without overlap.

Webcam Tab

The Webcam tab provides live, continuous frame-by-frame detection with no manual triggering required. After granting camera permission once, the system streams and annotates automatically at approximately 10 frames per second. The Auto-Night Mode toggle and the sensitivity slider are both directly accessible on this tab, allowing the operator to adjust detection behaviour in real time without navigating away. This tab is suited to continuous monitoring at a single checkpoint location.

Image Tab

The Image tab accepts a static image upload for full analysis. Results include the annotated image with bounding boxes and confidence labels, a morphological mask region visualisation showing the detected coverage areas, a per-face report with individual confidence scores and coverage percentages, and an overall compliance rate for all faces present in the image. This tab is suited to analysing captured frames from CCTV systems or reviewing individual photographs.

Video Tab

The Video tab processes a recorded video file frame by frame. The frame skip parameter is configurable from 1 to 5, allowing a trade-off between processing speed and annotation completeness. An annotated output video is generated and available for download, with each processed frame containing the same bounding boxes and labels as the live webcam view. A compliance summary is produced covering total faces detected and the overall mask compliance percentage across the full video duration. This tab is suited to retrospective compliance review of recorded footage from a shift or event.

CBIR Tab

The CBIR tab is the explainability view. Any face image can be uploaded to receive a classification result alongside the three most visually similar training examples displayed as a visual panel. This tab is designed for supervisors who need to understand or verify a classification decision before acting on it. It requires no technical knowledge to use and provides clear, visual justification for the system's output.

Because MobileNetV2 is a lightweight architecture originally designed for mobile deployment, and because the SVM pipeline runs entirely on CPU, the system does not require high-end server hardware for runtime inference. Once both models are trained, they can be served from a standard cloud instance or even a mid-range workstation. The Gradio interface generates a public share link automatically, meaning the system is accessible from any browser without any installation steps on the client side.

Deployment environments well suited to this system include hospitals and healthcare facilities monitoring staff and visitor compliance, transport hubs integrating with existing CCTV infrastructure, office buildings and factories with single-entry checkpoints, and educational institutions during health-sensitive periods. The video processing capability also extends the system's usefulness to retrospective analysis, where recorded footage from an event or shift is processed to produce a formal compliance report for regulatory or administrative purposes.

6. Economic Feasibility

The cost profile of the final system remains entirely within the open-source, zero-licensing model established at the mid-project stage. Every library used in the system, including PyTorch, TorchVision, OpenCV, scikit-learn, scikit-image, Gradio, NumPy, Matplotlib, SciPy, tqdm, and lxml, is freely available under permissive open-source licences. There are no subscription fees, API costs, or per-inference charges associated with any component of the stack.

Training was conducted on Google Colab's free-tier NVIDIA T4 GPU. The full training run of 20 epochs over approximately 3,200 training patches at 128x128 resolution completed within a single Colab session without hitting the free-tier time limit. The dataset is approximately 150 MB in size and was downloaded directly via the Kaggle API at no cost.

For production deployment, the system can be hosted on any standard cloud compute instance. MobileNetV2 inference is fast enough for real-time operation on a single CPU instance at typical entry-point throughput. GPU hardware is only required during the initial training phase, which is a one-time cost. After training is complete, the serialised model files are all that is needed for deployment.

The economic case for this system is straightforward. The recurring cost of an automated AI-based compliance monitoring system is a fraction of the cost of equivalent human monitoring personnel, particularly in scenarios requiring 24-hour coverage or monitoring across multiple zones simultaneously. The system also eliminates the variability and fatigue that affect human monitors over extended shifts. Given that all development tooling, all runtime libraries, and the training compute are available at zero cost, the total cost of ownership is limited to the infrastructure needed to host the Gradio interface, which can be as minimal as a single shared cloud instance.

7. Schedule Feasibility

The final project was developed incrementally across the Computer Vision Lab sessions, with each development phase building directly on the work completed in the previous one. The table below maps each phase to the specific work completed and the approximate duration in lab session time.

Phase	Work Completed	Duration
Phase 1: Dataset and Baseline	Kaggle dataset download via API, PASCAL VOC XML annotation parser, face patch extraction from bounding boxes, HOG feature extraction, initial SVM training and evaluation	2 weeks
Phase 2: Feature Enhancements (Lab 07)	Gabor filter bank with 4 orientations and 4 frequencies (16 kernels), HSV colour histogram extraction, three-way feature fusion into a single vector per patch	2 weeks
Phase 3: PCA Pipeline and CBIR (Labs 08, 11, 12)	StandardScaler plus PCA(100) plus SVM sklearn Pipeline, cosine-distance CBIR patch index construction, CBIR visual retrieval UI tab in Gradio	2 to 3 weeks
Phase 4: Deep Learning (MobileNetV2)	MobileNetV2 fine-tuning with partial layer freeze, weighted CrossEntropyLoss for class imbalance, AdamW with CosineAnnealingLR scheduler, 20-epoch training with best-model checkpointing	2 weeks
Phase 5: Interface, Night Mode, and Testing	Auto-Night Mode with adaptive gamma LUT, morphological mask-coverage analysis pipeline, full four-tab Gradio interface, video file processor, final integration testing and cleanup	2 weeks

The use of pretrained MobileNetV2 weights through transfer learning was a critical scheduling decision. Training a comparably capable model from random initialisation would have required several hundred epochs and multiple full Colab sessions. Transfer learning allowed the model to converge in 20 epochs in a single session by reusing the low-level feature detectors already learned from ImageNet. Each of the five phases corresponds directly to identifiable lab topics covered during the semester, meaning the development timeline stayed aligned with the course schedule throughout. No phase required significant rework of prior phases, which reflects the modular architecture of the system.

8. Conclusion

MaskGuard AI is a fully functional, multi-modal face mask detection system that successfully combines classical computer vision techniques and deep learning into a single deployable application. The final version addresses every limitation identified in the mid-project baseline.

Class imbalance across the three label categories is handled through weighted loss in MobileNetV2 training and balanced class weights in the SVM, ensuring the minority classes receive appropriate training attention. The feature representation is significantly richer than the mid-project baseline, combining HOG, Gabor texture, and colour histogram descriptors into a single fused vector. False positives in face detection are reduced by rationalising the cascade pipeline from four simultaneous classifiers to two, with CLAHE preprocessing and a user-adjustable sensitivity control. Predictions are made explainable through the CBIR retrieval tab. Real-world usability is improved through the Auto-Night Mode, the video processing pipeline, and the well-structured four-tab interface.

All technical, operational, economic, and schedule feasibility criteria are met. The system uses no proprietary or paid tools, runs on a free Colab GPU for training, and can be deployed on standard infrastructure for inference. The codebase is structured, documented, and available on GitHub. MaskGuard AI is a complete, production-ready academic deliverable with clear real-world utility across healthcare, transport, commercial, and educational environments.

9. Team Collaboration

Team Roles and Responsibilities

Team Member	Role	Responsibilities
Data Fardeen	Code Development	Implemented the full system including dataset pipeline, HOG/Gabor/PCA feature engineering, CBIR index, MobileNetV2 fine-tuning, morphological analysis, Auto-Night Mode, and the Gradio interface
Iman Humayun	Feasibility Study	Conducted feasibility analysis and prepared the feasibility study documentation
Amina Asghar	Presentation Design	Prepared and organized the PowerPoint presentation and project slides

Contribution Matrix: Task Distribution

Project Task	Data Fardeen	Iman Humayun	Amina Asghar
Dataset Preparation and Annotation Parsing	Yes		
HOG + Gabor + Colour Feature Fusion	Yes		
PCA Pipeline and SVM Classification	Yes		
CBIR Retrieval Index (Lab 08)	Yes		

Project Task	Data Fardeen	Iman Humayun	Amina Asghar
MobileNetV2 Fine-Tuning and Training	Yes		
Morphological Mask Coverage Analysis	Yes		
Auto-Night Mode (Gamma Correction)	Yes		
Gradio Interface (4-Tab)	Yes		
Feasibility Study Writing		Yes	
Documentation		Yes	
PowerPoint Preparation			Yes
Final Review	Yes	Yes	Yes

GitHub Repository: https://github.com/Fardeen37/Face-Mask-Detection-System